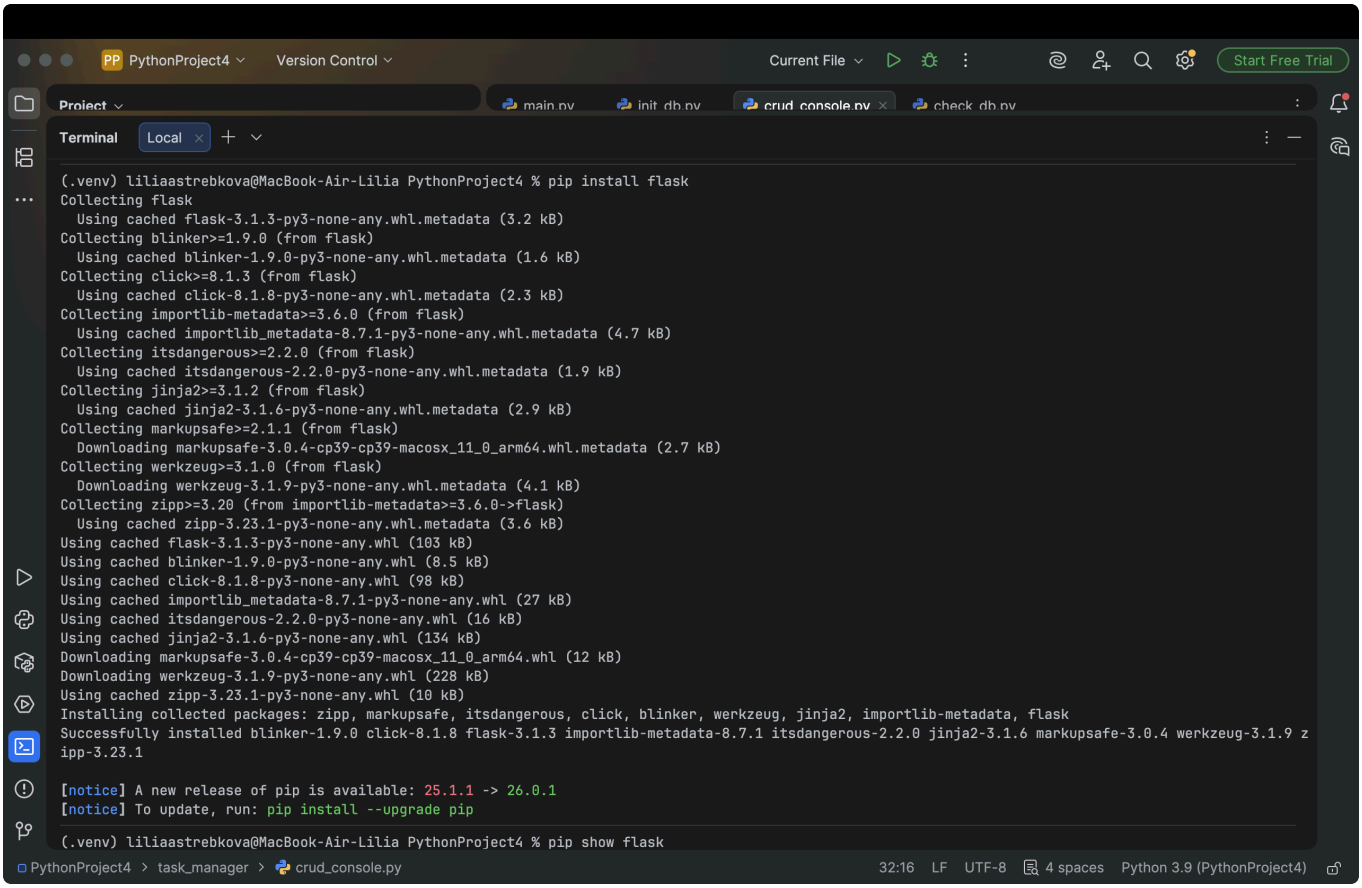
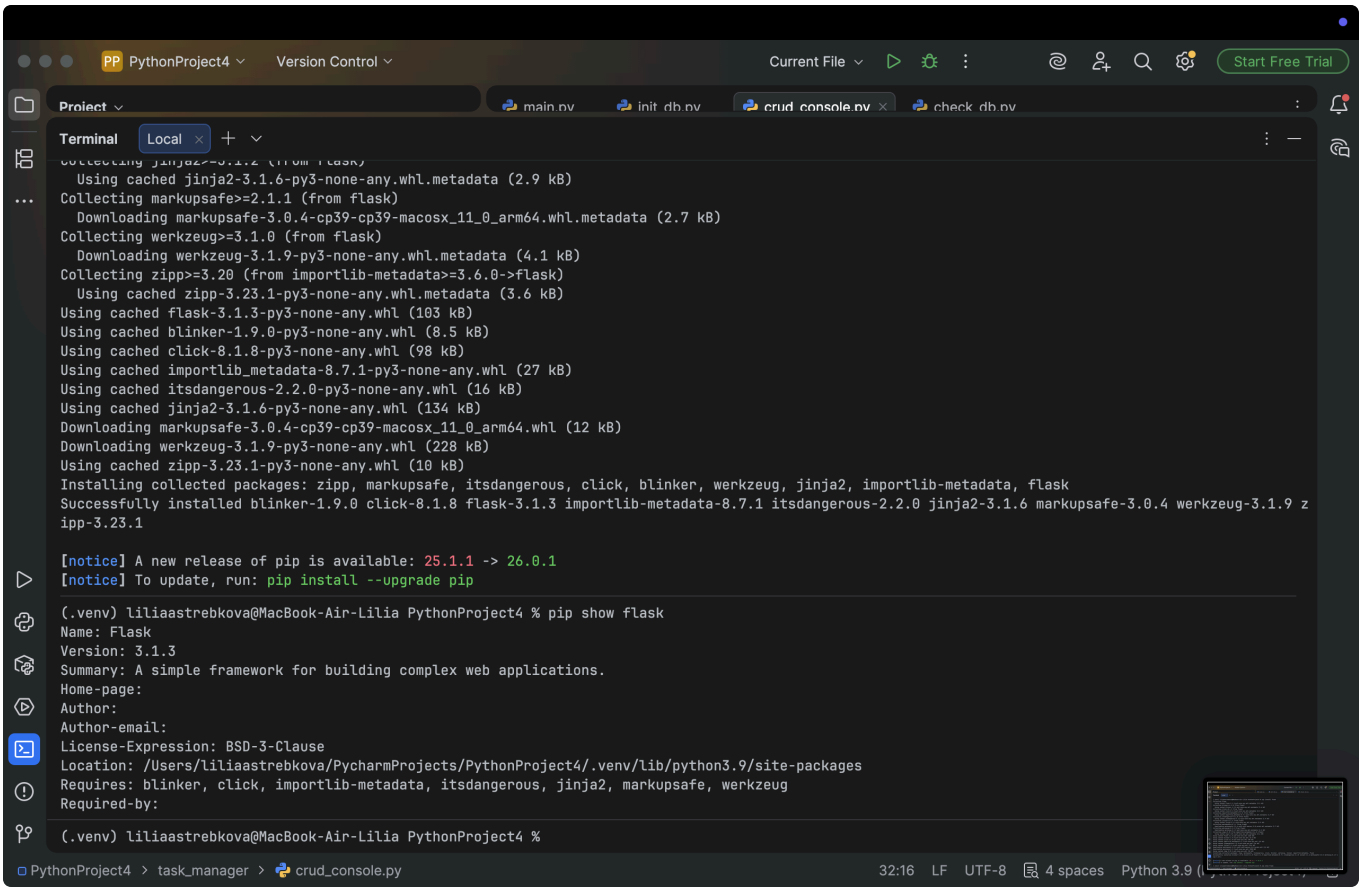
Подключение Flask и первый маршрут (Hello World)

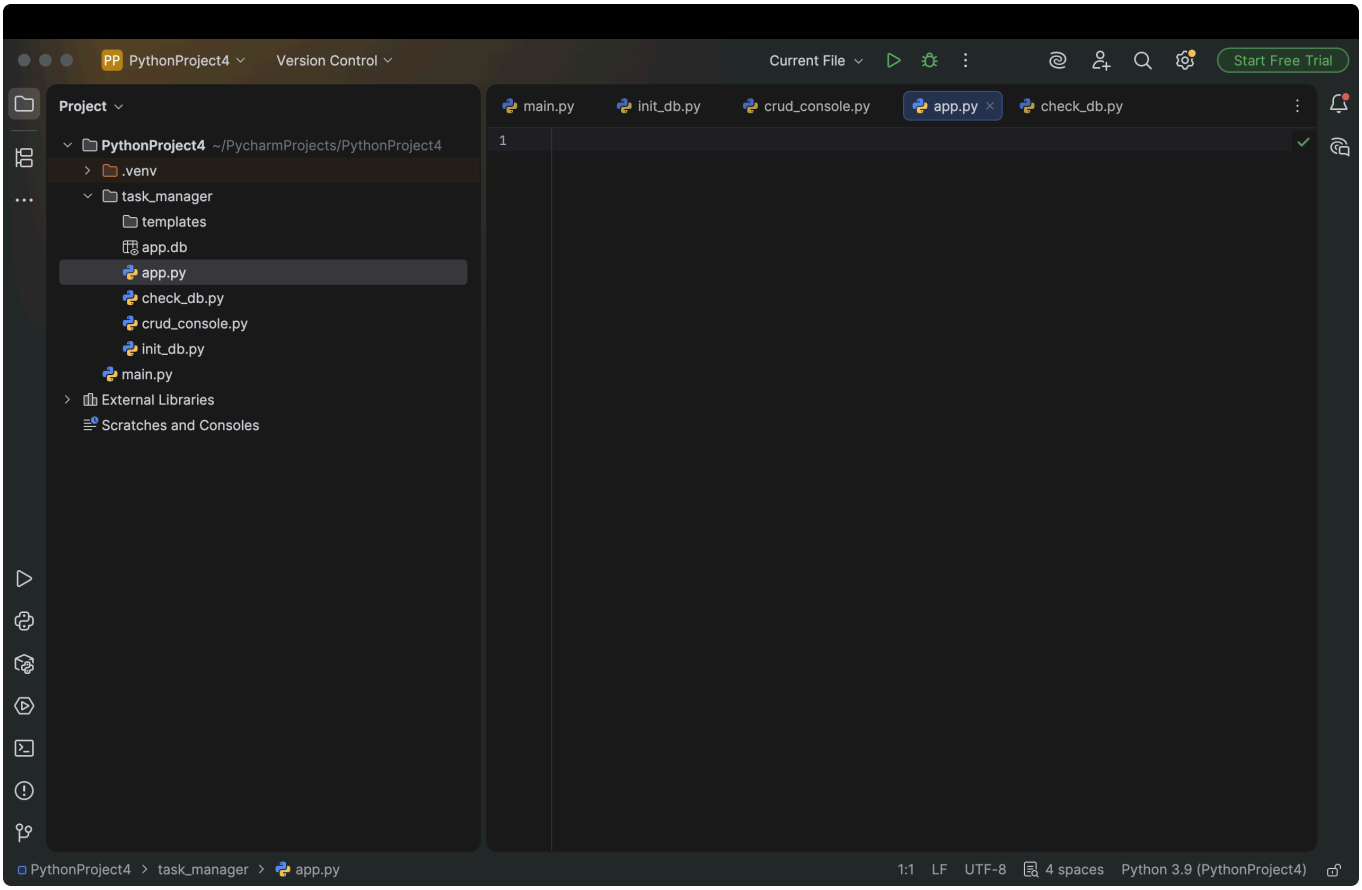
Ястребкова Лилия ЗИСП11-44

Шаг 1. Установка Flask

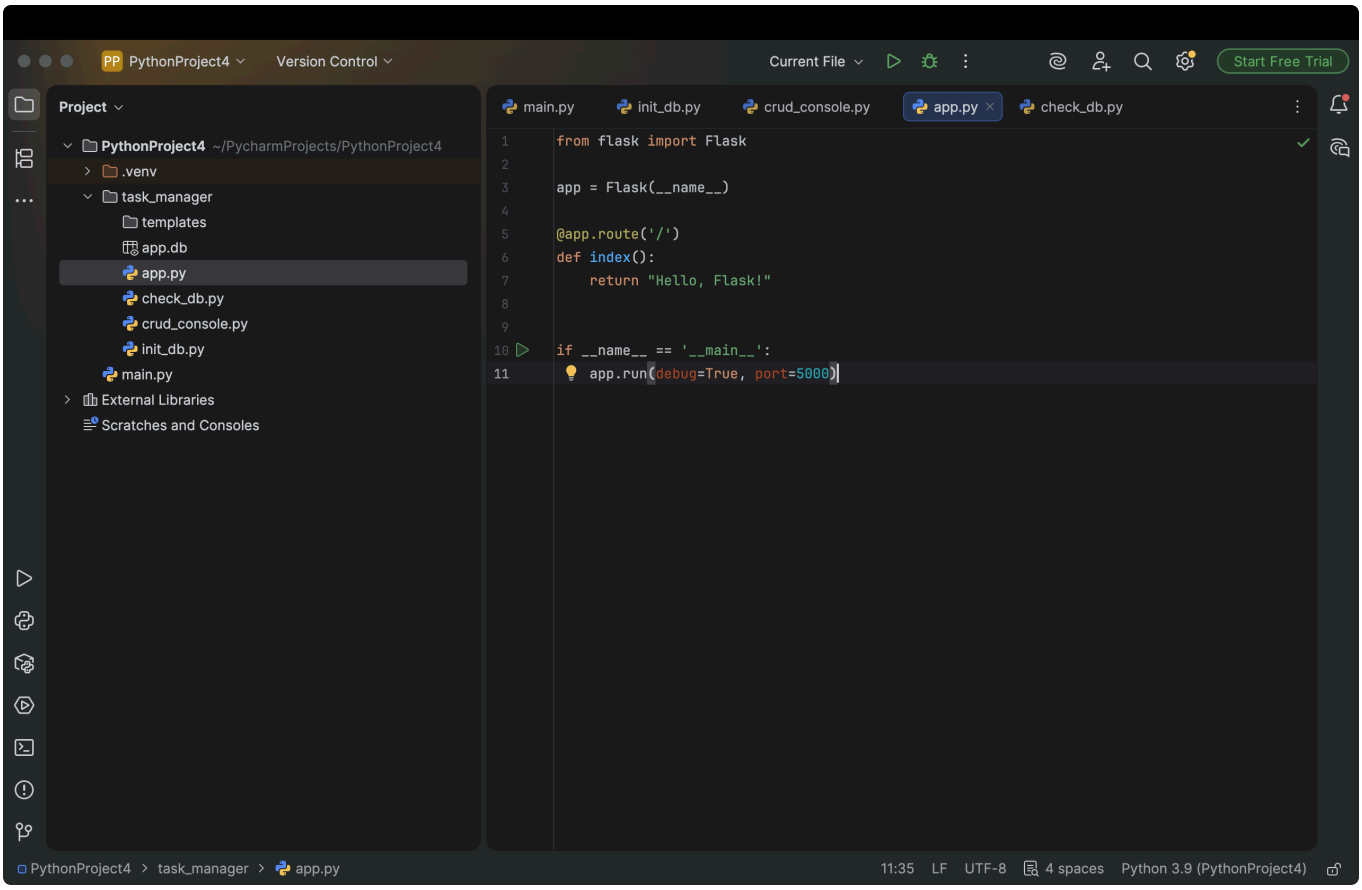




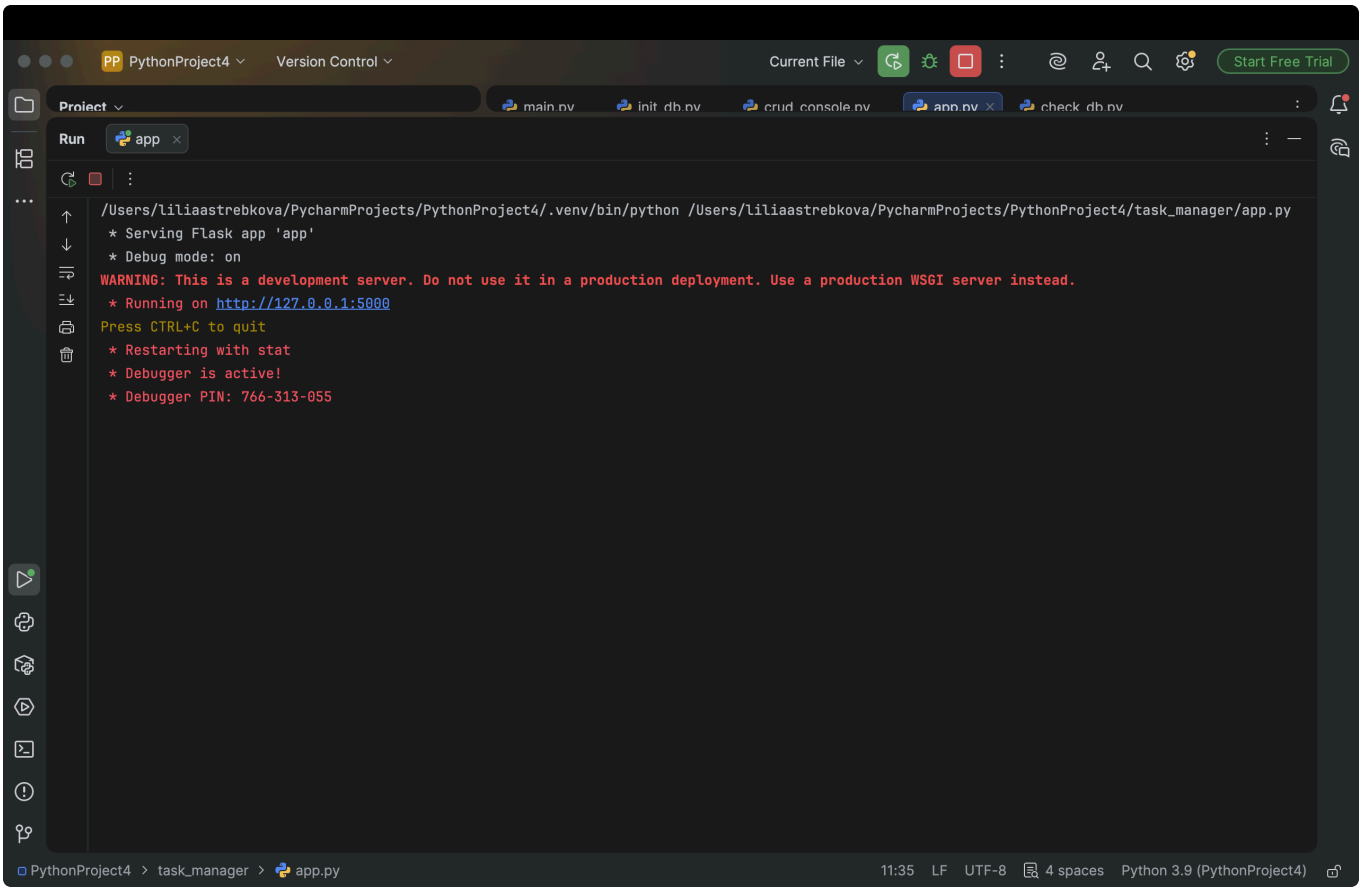
Шаг 2. Структура проекта



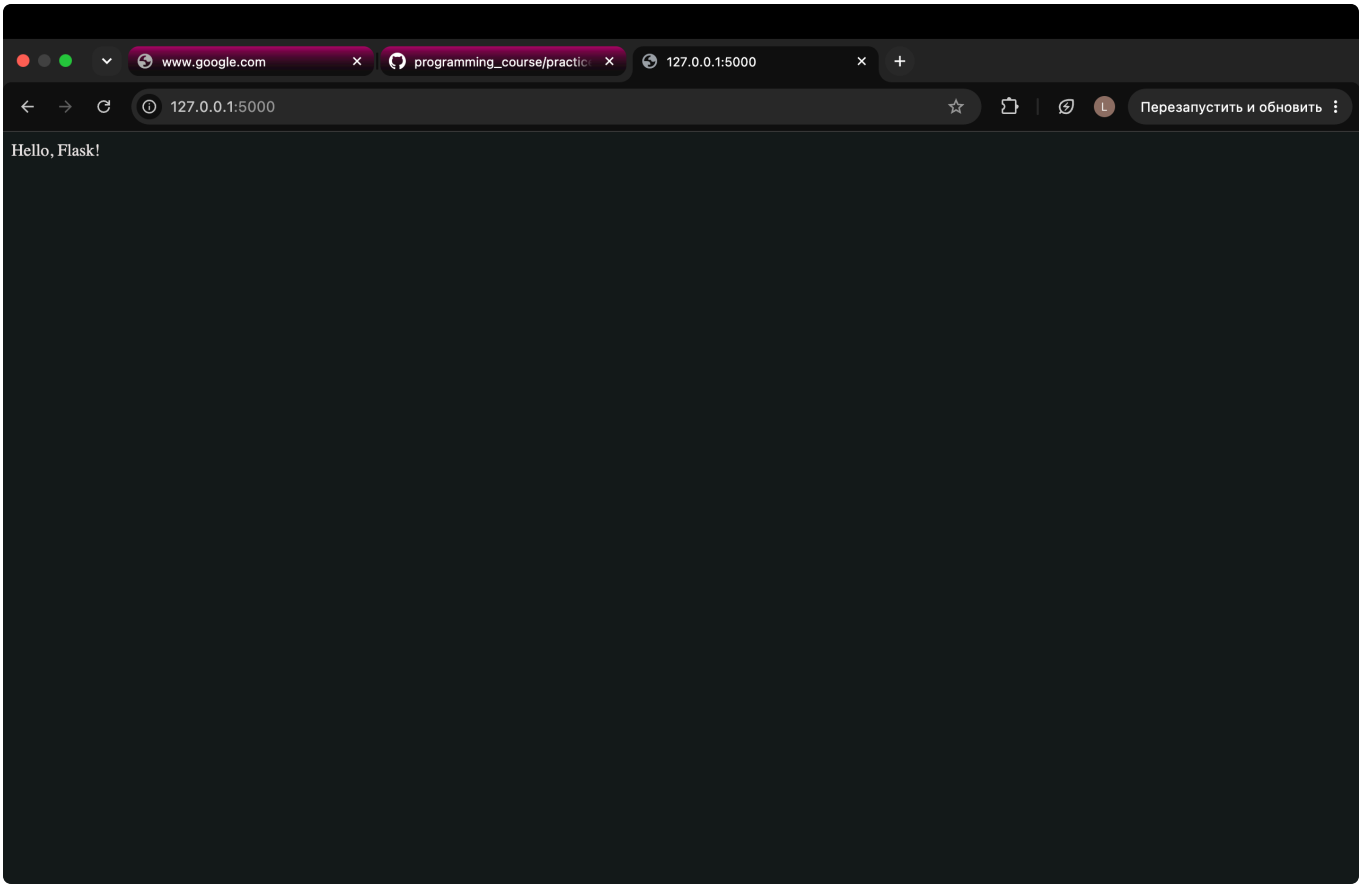
Шаг 3. Первый маршрут — «Hello, Flask!»



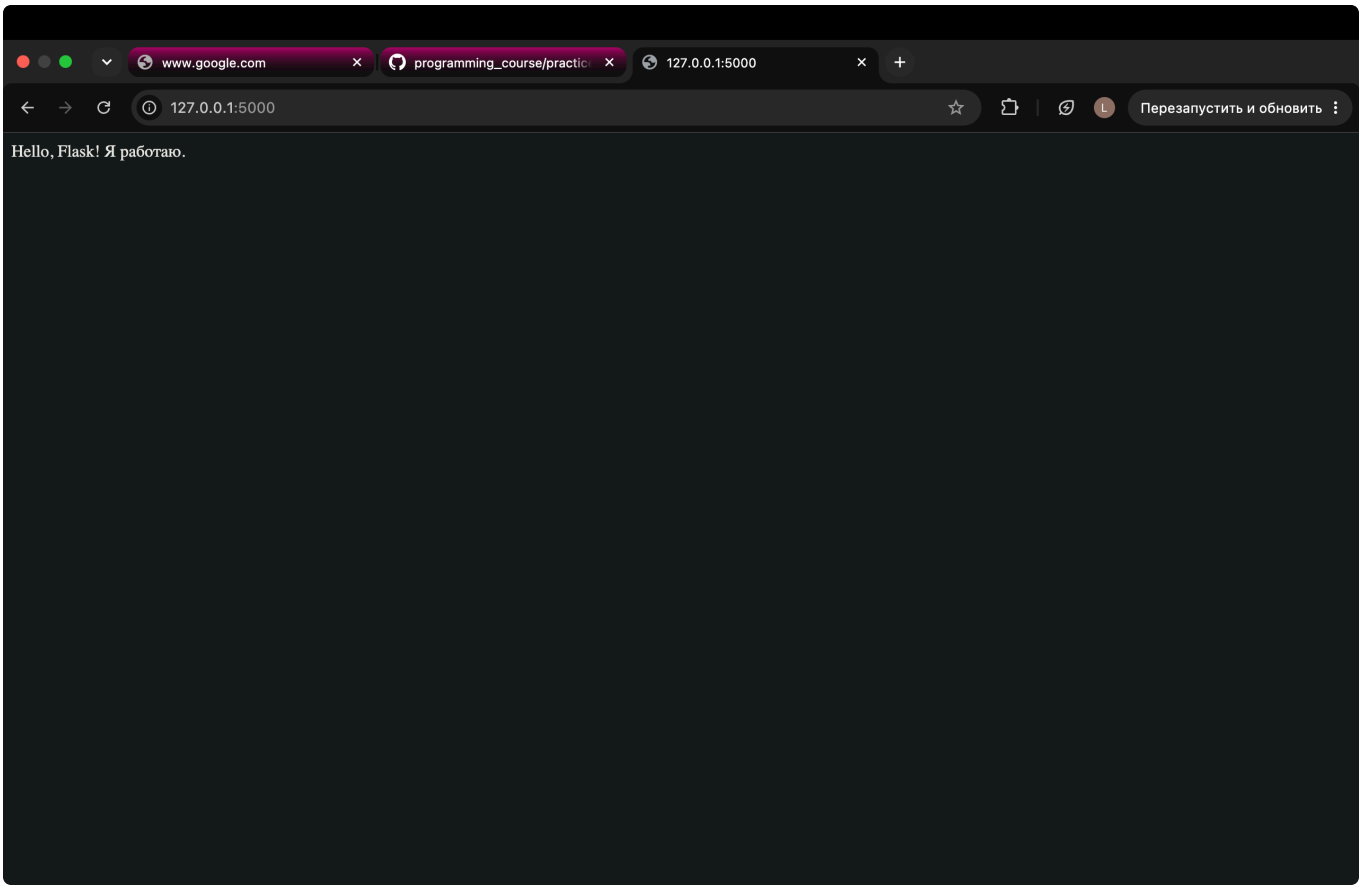
Шаг 4. Запуск сервера



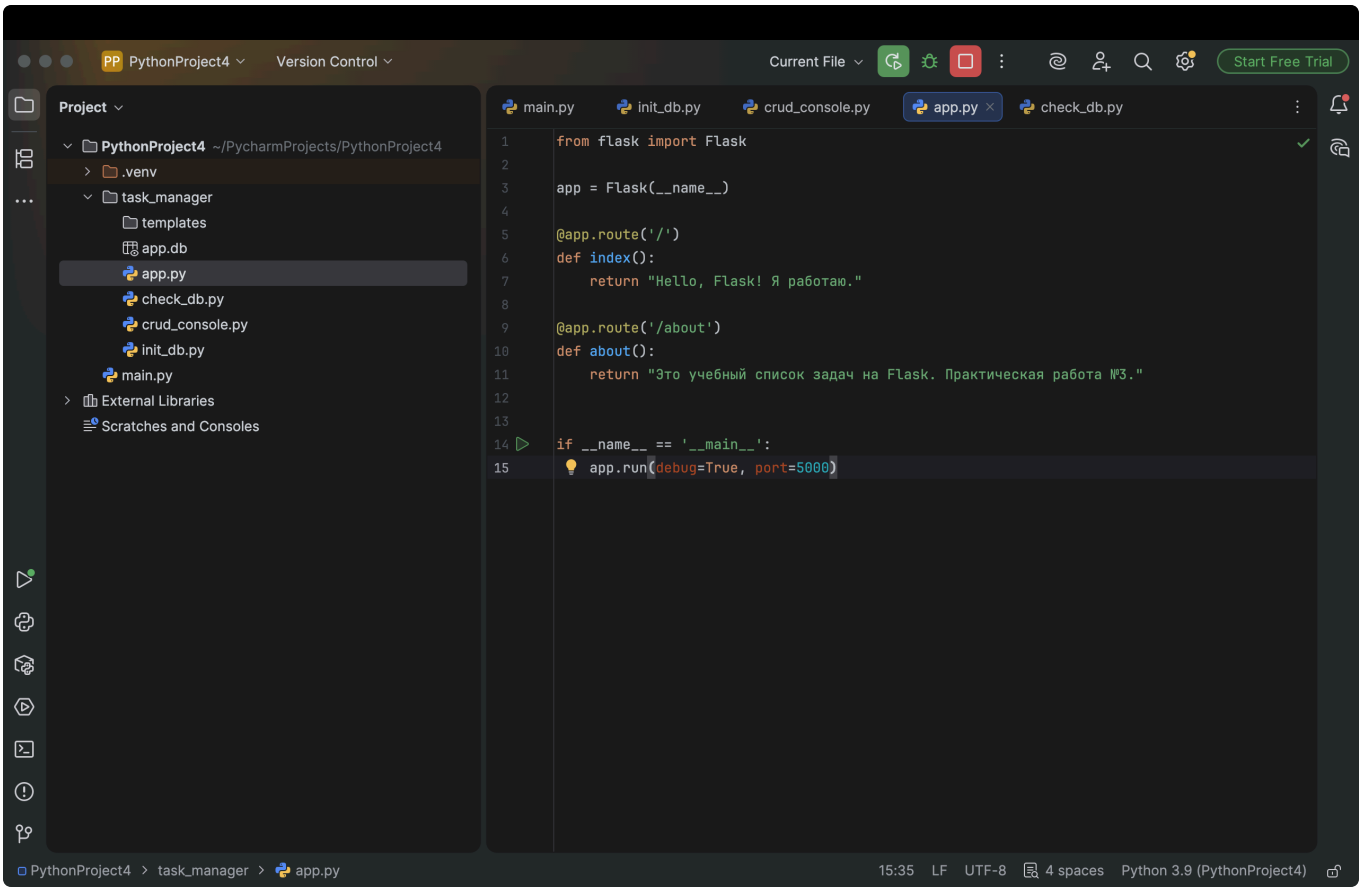
Шаг 5. Открытие в браузере



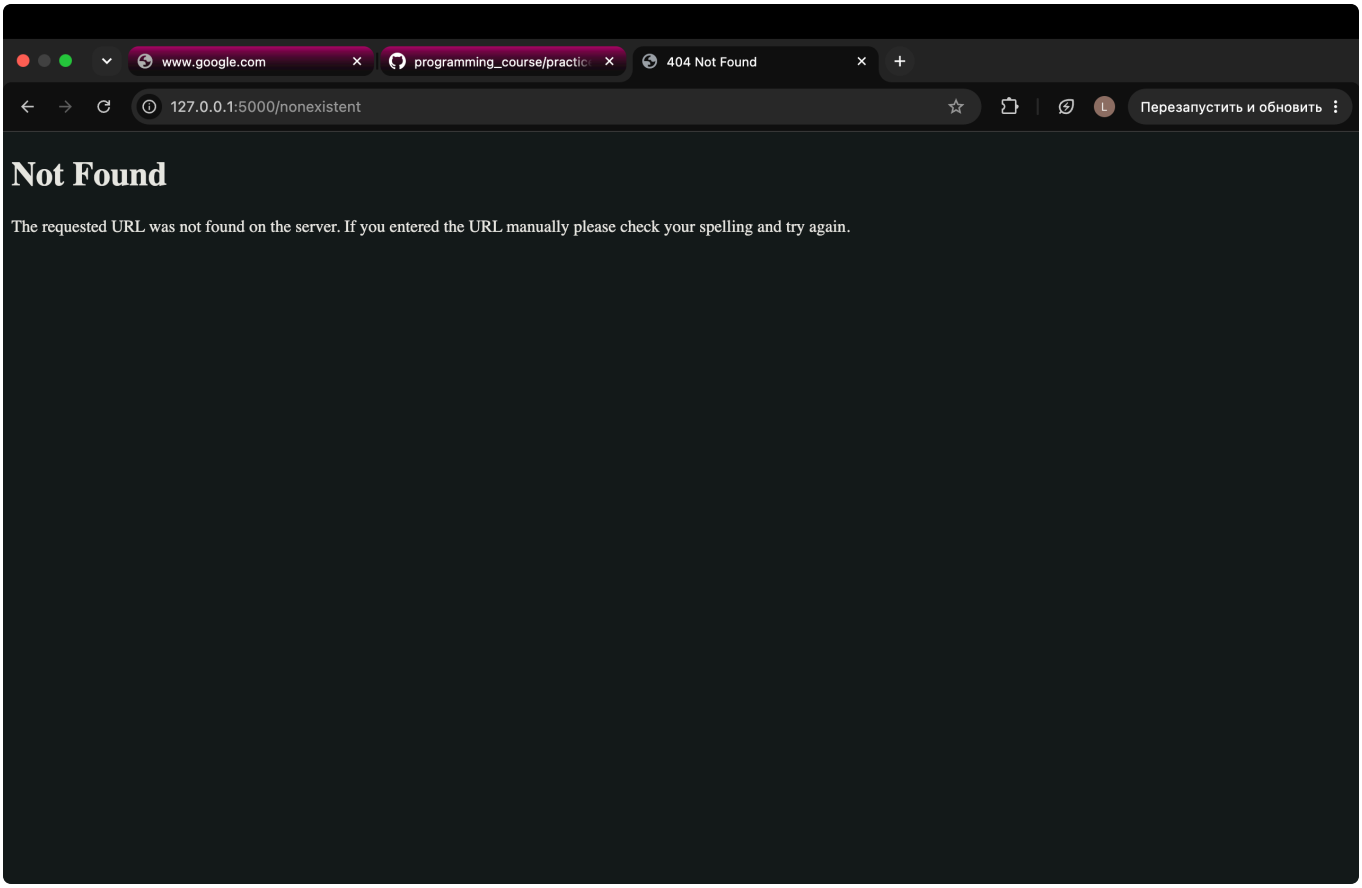
Шаг 6. Проверка автоперезагрузки



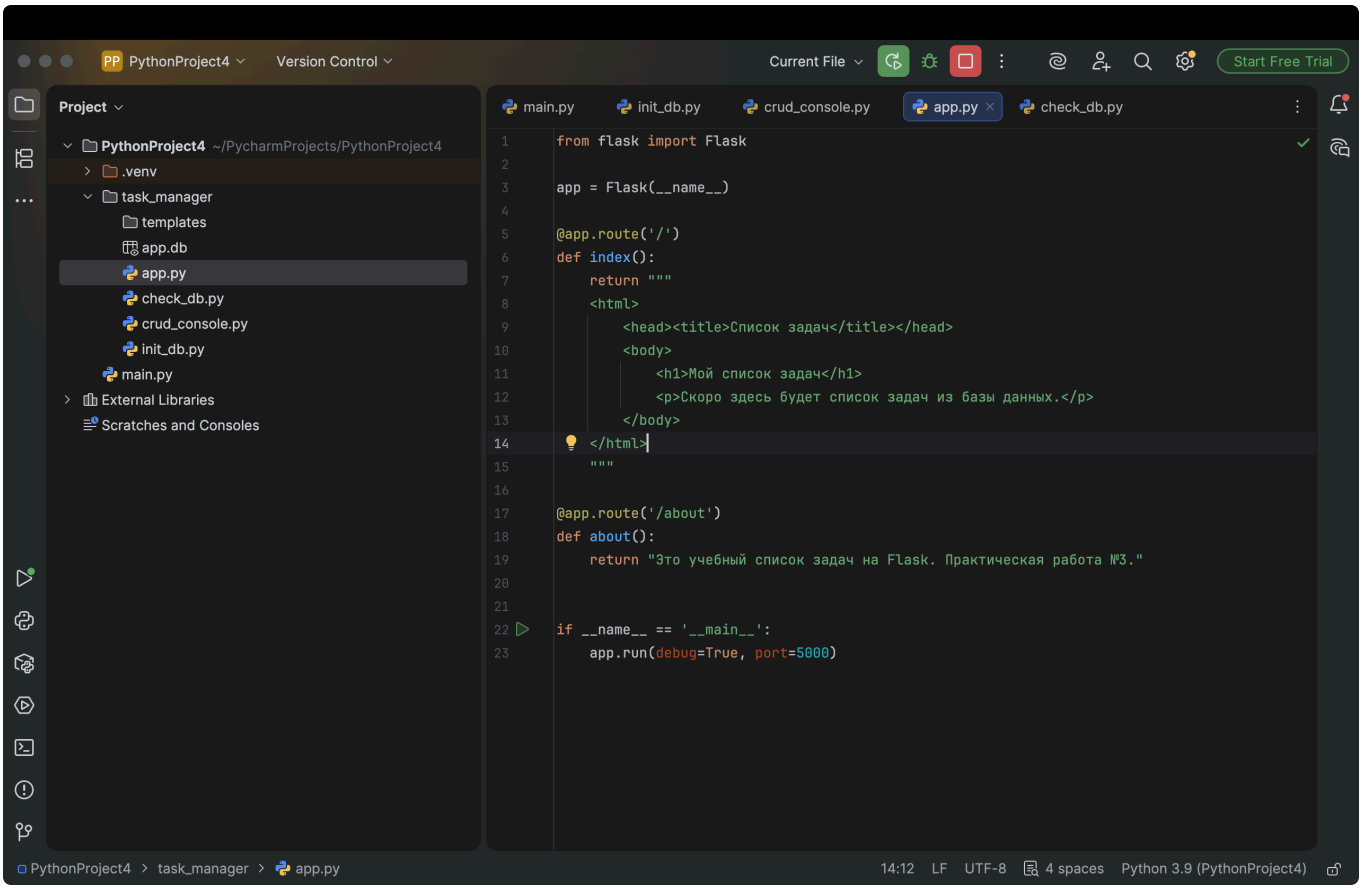
Шаг 7. Второй маршрут — добавление страницы

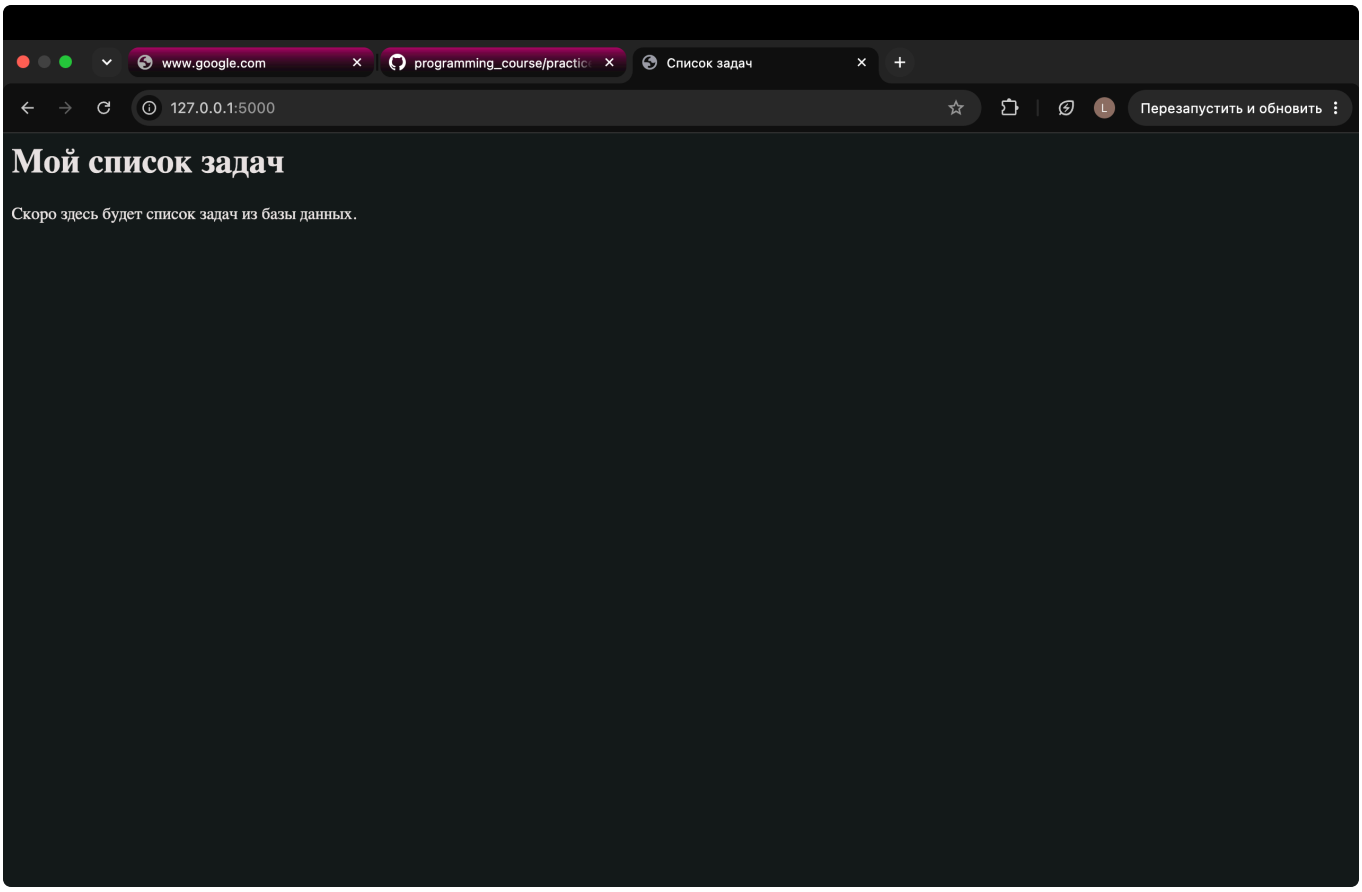


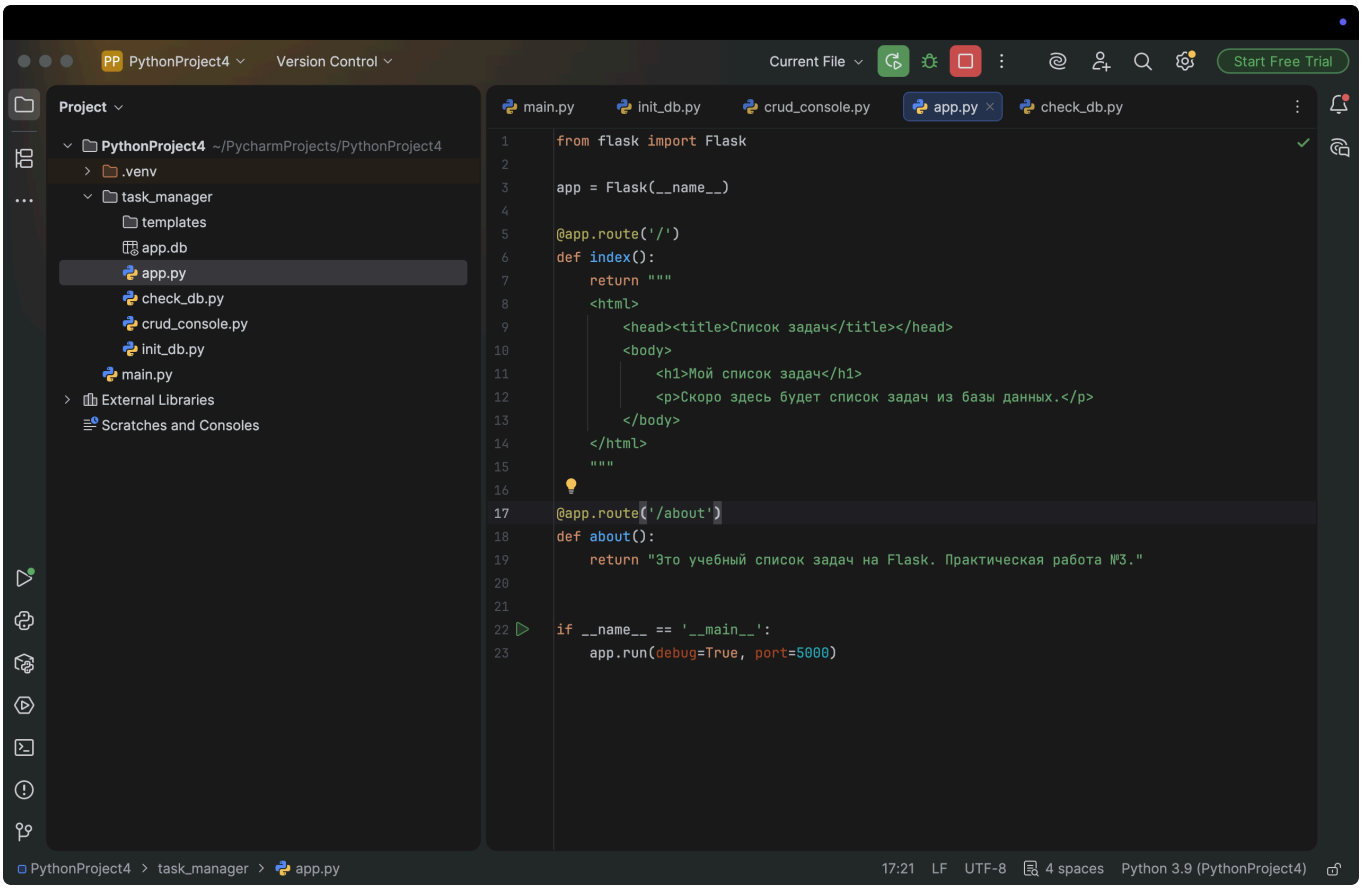
Шаг 8. Проверка обработки ошибок 404



Шаг 9. Возврат HTML-строки (первое приближение к веб-странице)







1. Что делает декоратор `@app.route('/')`?

Он регистрирует функцию как обработчик маршрута: Flask запоминает, что при запросе браузера к URL `/` нужно вызвать функцию, стоящую под декоратором (`index()`), а её возвращаемое значение отправить браузеру как HTTP-ответ. То есть декоратор связывает URL с функцией-обработчиком (view function)

2. Чем `GET` отличается от `POST`? Какой метод используется по умолчанию?

`GET` - запрос «дай мне страницу»: параметры передаются в самом URL, используется для получения данных и открытия страниц.
`POST` - запрос «прими и обработай данные»: они передаются в теле запроса, используется для отправки форм, создания и изменения данных. По умолчанию маршрут во Flask принимает только `GET`; чтобы принимать `POST`, нужно указать это явно: `@app.route('/', methods=['GET', 'POST'])`

3. Зачем нужен `debug=True` и почему его нельзя использовать на реальном сервере?

Режим отладки даёт два преимущества: автоматическую перезагрузку сервера при изменении кода (не нужно останавливать и запускать вручную) и подробные сообщения об ошибках с трассировкой прямо в браузере - это ускоряет разработку. На рабочем сервере его включать нельзя, потому что страница с отладочной информацией открывает внутренности приложения: пользователь может выполнять произвольный код через интерактивную консоль отладчика, а также видеть исходный код и переменные. Это прямая угроза безопасности, поэтому в продакшене используют другие способы запуска (например, WSGI-сервер) и логирование ошибок без вывода в браузер.

4. Что означает адрес `127.0.0.1:5000`?

`127.0.0.1` - это адрес localhost, то есть самого компьютера: сервер поднят локально, и доступ к нему есть только с этой же машины. `5000` - порт, который слушает Flask: это «дверь», через которую принимаются запросы. Вместе они задают полный адрес сервера: `http://127.0.0.1:5000`

5. Почему при открытии `/nonexistent` мы получаем 404, а не ошибку Python?

Flask сам сопоставляет запрошенный URL с зарегистрированными маршрутами. Если маршрута нет, он не вызывает ни одну view-функцию (значит, Python-код не выполняется и исключение не возникает), а возвращает стандартный HTTP-ответ с кодом 404 Not Found - «ресурс не найден». Это предусмотренное поведение фреймворка, а не сбой программы.

6. Почему папка для HTML-шаблонов должна называться именно `templates`?

Это соглашение Flask, зашитое по умолчанию: при создании приложения `Flask(__name__)` фреймворк ищет шаблоны в папке `templates` рядом с файлом приложения. Если назвать папку иначе, `render_template()` не найдёт файл и выбросит исключение `TemplateNotFound`. (Имя можно изменить через параметры конструктора, но стандартный проект так не делает.)

7. Что произойдёт, если запустить два сервера Flask одновременно на одном порту?

Один порт на одном компьютере может занимать только один процесс. Второй сервер не сможет забрать порт 5000 и упадёт с ошибкой `OSError: [Errno 98] Address already in use` (в Windows - `OSError: [WinError 10048]`). Чтобы запустить оба, нужно либо остановить первый (`Ctrl+C`), либо указать второму другой порт, например `port=5001`.